Responsive Web Design

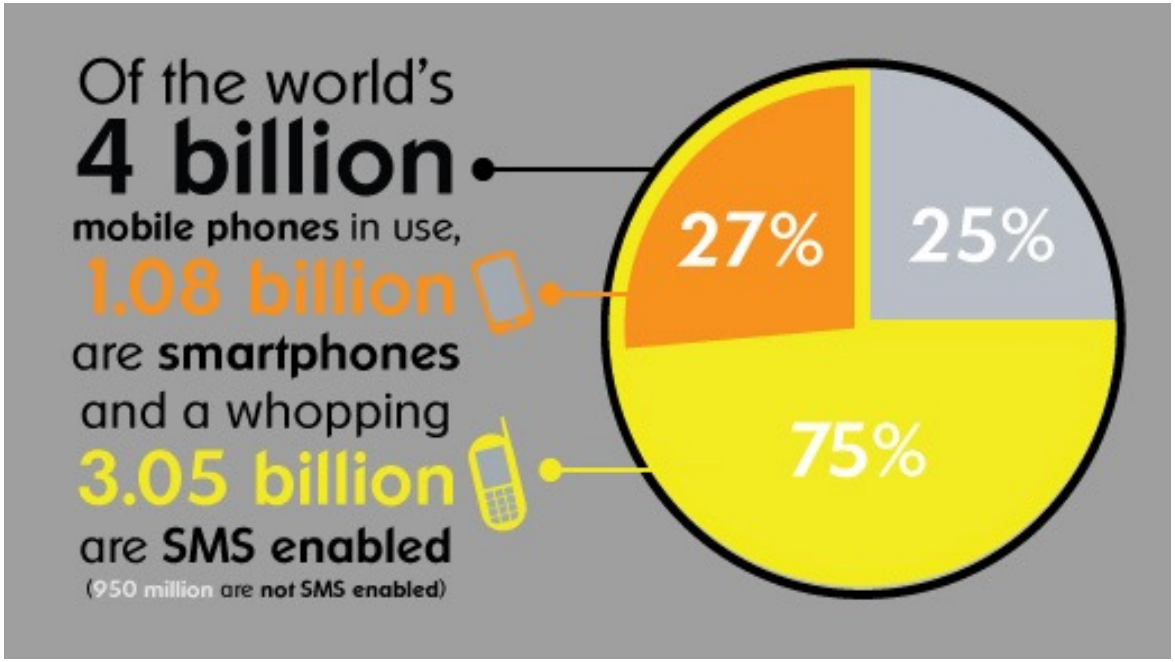
What are we Talking about today?

- Key Concepts of Responsive Design
  - Fluid Grids
  - Flexible Images
  - CSS Media Queries
- Mobile-First Design Approach
  - Focus on Content
  - Optimizing User Experience
  - Avoiding Separate Mobile Sites
- Tools and Techniques
  - Viewport Meta Tag
  - Frameworks
  - Cross-Browser Compatibility

Outline

- The Need - Mobile Growth
- What is Responsive Web Design
- How to Design Responsively
- Major Technology Features
  - media queries
  - fluid grids
  - scalable images
- More Examples of Responsive Websites

Size of the Mobile Market

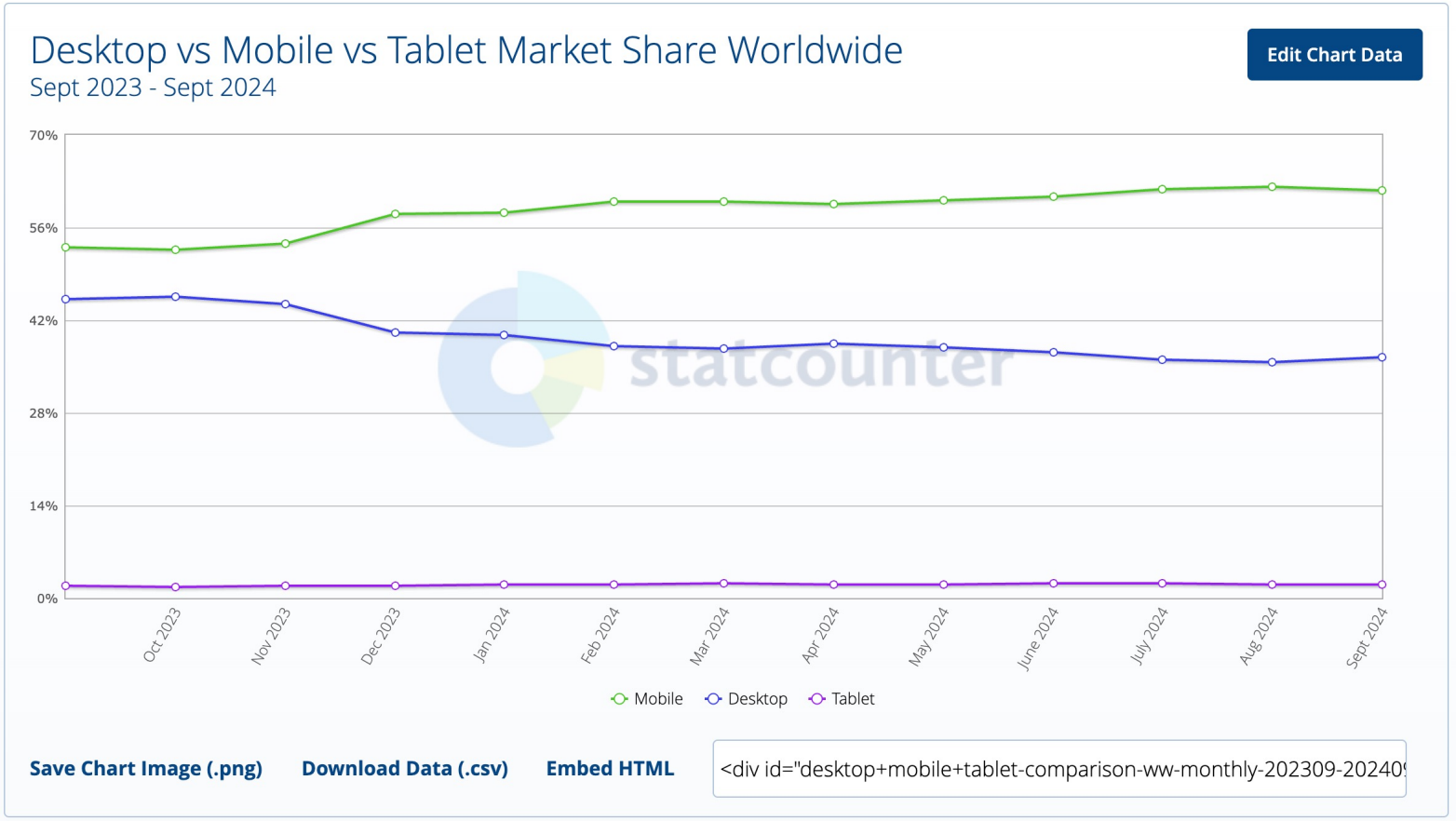


The Growth of Mobile Marketing and Tagging by Microsoft Tag

- There is a proliferation of mobile devices worldwide.
- More and more people access Internet only through mobile devices.
- Estimated by the year 2020, 12 billion mobile subscriptions.
- Websites must be designed to make sure the mobile viewer has an excellent experience.

How Fast is the Mobile Internet Growing?

- From http://gs.statcounter.com/platform-market-share/desktop-mobile-tablet



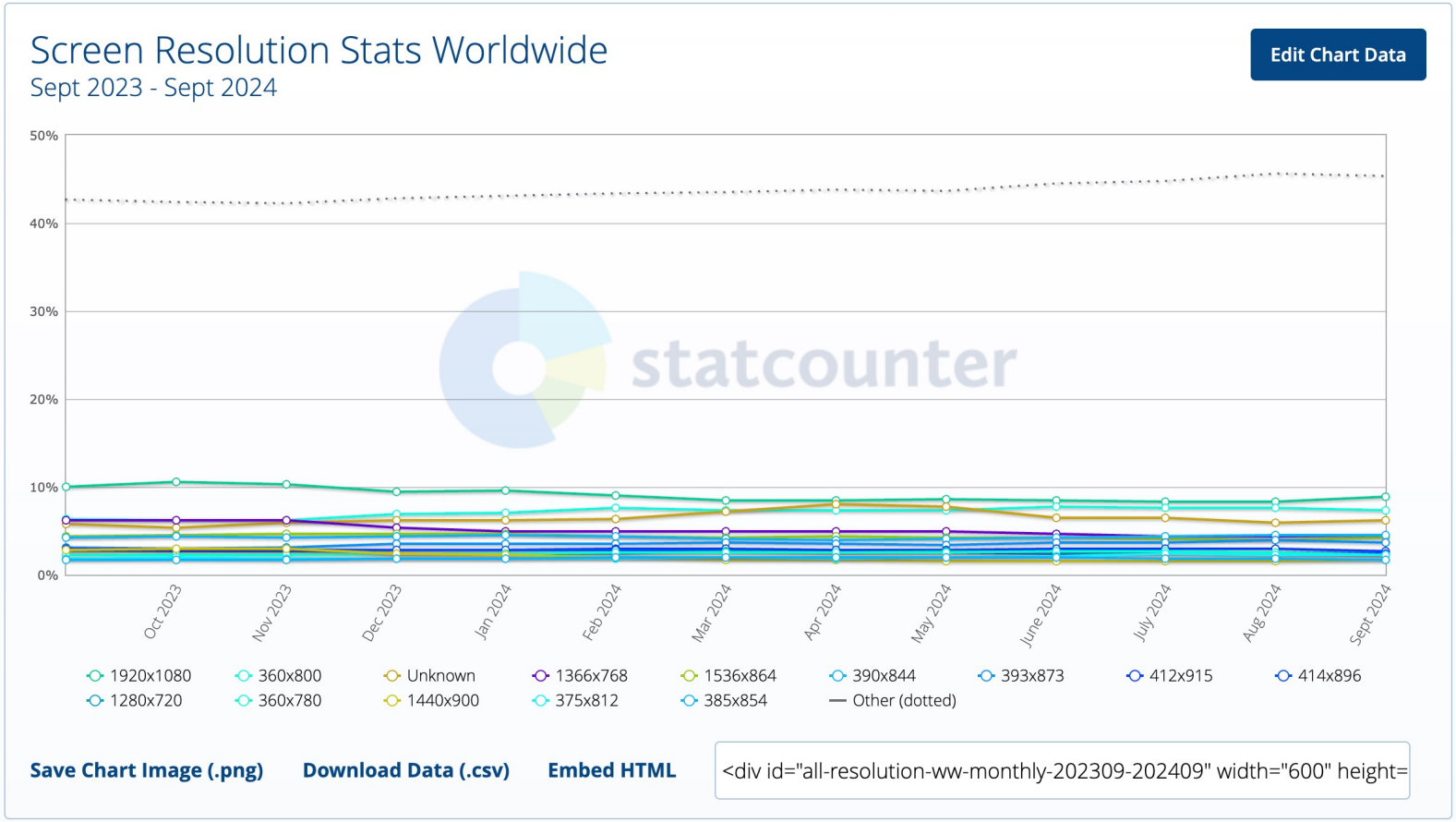
As of October 2020, mobile internet had overtaken desktop internet usage.

Mobile Forces You to Focus

- Requires the design team to focus on the most important content, data, and functions.
- There is no space in a 320 by 480-pixel screen (the original 2007 iPhone) for extraneous, unnecessary elements
- More recent devices have resolutions of 1,000 ppi, but the screen size is still between 4 and 6 inches
- One is forced to focus on content that's most important to users
  - What features and functionality are essential
  - It may be nice to have, but does it belong on your page
- Designers must prioritize

Screen Resolution Stats

- From http://gs.statcounter.com/screen-resolution-stats



Building for Mobile Can Extend Your Capabilities

- The design technique of "Mobile first" encourages designers to use the full capabilities of the mobile device.

"Saying mobile design should have less is like saying paperbacks have smaller pages, so we should remove chapters." (Clark)

- Mobile devices offer
  - Use of **geo-location** to optimize the experience.
  - Require **Switch layouts** depending on the way they're held.
  - Need to support rich, **multi-touch** interfaces
    - input devices that recognize two or more simultaneous touches
      - e.g., two finger tap, two finger scroll, pinch, zoom
    - some devices also recognize differences in pressure and temperature (i.e., Apple Watch)

Design for the Mobile Web

- There are three main approaches:
  1. Build an entirely separate mobile **.mobi site**
     - The domain name **mobi** is a top-level domain. Its name is derived from the adjective *mobile*, indicating it is used by mobile devices for accessing Internet resources via the Mobile Web.
     - The domain was approved by ICANN on 11 July 2005, and is managed by the mTLD global registry
     - To date only 0.06% of web TLDs:

     http://w3techs.com/technologies/details/tld-mobi-/all/all
  2. Host the mobile site within your current domain (a **subdomain**) (**mobile.**mycompany.com)
  3. Configure your current website for mobile display using *Responsive Web Design* (**RWD**) techniques

Reasons for not using mobile.mycompany.com websites

1. Redirects can hinder/annoy search engines
2. Redirects take lots of time
3. If you offer a mobile.website for iPhone, what about for iPad, Android, etc.
4. Sharing a mobile.website will not work for people on laptops, as they will end up with a site designed for a small screen
5. Philosophical: every web resource should live at one URL!

In conclusion, building a *single responsive website* is the preferable way to go.

Responsive Web Design

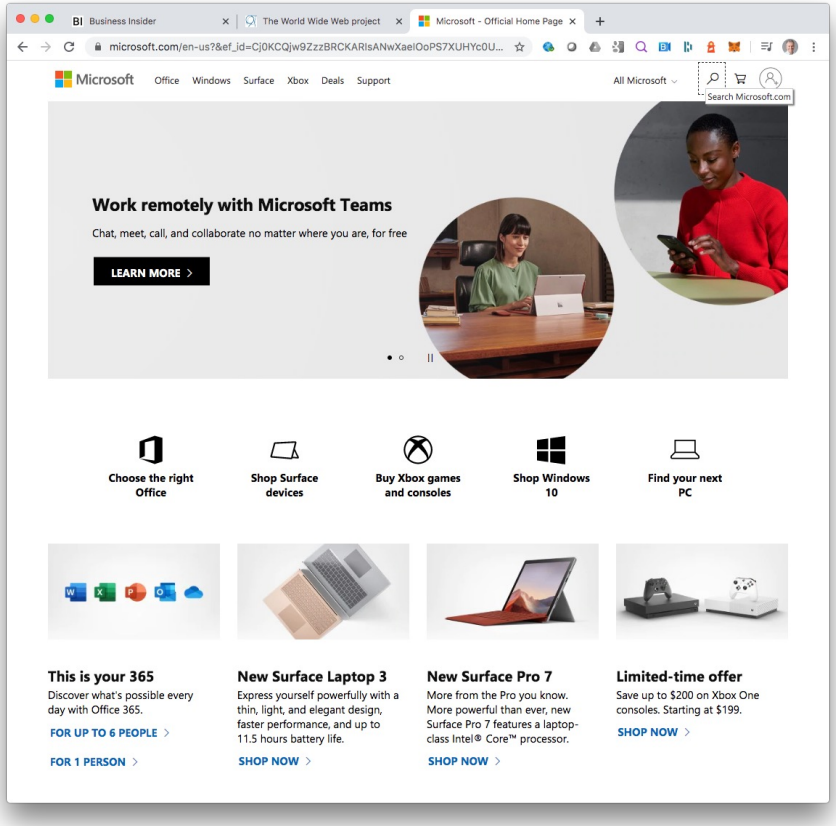
- **RWD** is the concept of developing a website in a way that **allows the layout to automatically adjust** according to the user's screen resolution (called its *viewport*).
- The viewport meta tag lets you set the width and initial scale of the viewport.
- For example:

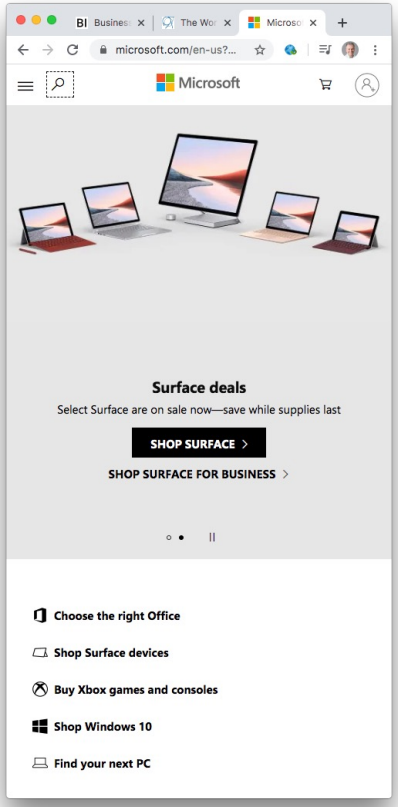
```
<meta name="“viewport” content="“width=590”">
```

- See viewport sizes at

http://viewportsizes.com/

Responsive Web Site Example: Microsoft





Use your laptop/ desktop, tablet and smartphone to check out the website

https://microsoft.com/

Popular Viewport Sizes

• Smartphones:	**Size (pixels)**	**Pixels/in**	**Size**
iPhone 16 Pro Max	2868x1320	460 ppi	6.9in
iPhone 12/13/14 Pro Max	2778x1284	458 ppi	6.7in
iPhone X/XS/11	2436x1125	458 ppi	5.8in
iPhone 7/8 Plus	1920x1080	401 ppi	5.5in
Samsung Galaxy S24 Ultra	3120x1440	505 ppi	6.8in
Samsung Galaxy S23 Ultra	3088x1440	500 ppi	6.8in
• Tablets:			
iPad Air 2	2048x1536	264 ppi	9.4in
iPad mini 3	2048x1536	326 ppi	7.9in
Samsung Galaxy Tab S	1600x2560	360 ppi	8.4in, 10.5in
• Notebooks:			
MacBook Air	1440x900	128 ppi	13.3in
MacBook Pro Retina	2880x1800	220 ppi	15.4in
• Desktops:			
iMac	2560x1440	109 ppi	27in

Responsive Web Design

- The term "Responsive Web Design" was coined by **Ethan Marcotte** in an article on May 25, 2010.
  - for his article see http://www.alistapart.com/articles/responsive-web-design
  - for his website see http://ethanmarcotte.com/
- Responsive web design (RWD) is a web design approach that tries to achieve an ideal viewing experience, which means:
  - easy reading and navigation with a minimum of resizing, panning, and scrolling
  - across a wide range of devices (from mobile phones to desktop monitors)
- A site designed with RWD adapts the layout to the viewing environment by using
  1. **fluid**, proportion-based **grids**,
  2. **flexible images**, and
  3. **CSS3 *media queries*** (you provide different CSS based upon the viewport size).
     - see the W3C documentation of Media Queries at

     http://www.w3.org/TR/css3-mediaqueries/

Media Queries

- W3C created *media queries* as part of the CSS3 specification.
  - http://www.w3.org/TR/css3-mediaqueries/
- A *media query* consists of a *media type* (`screen`) and the actual query enclosed within parentheses, containing a particular *media feature* (`max-device-width`) to inspect, followed by the *target value* (`480px`). The query can be zero or more expressions that check for the conditions of particular media features
  - **Example below**: Style sheet (example.css) applies to devices of a certain media type (screen) with certain feature (color screen).

```
<link rel="stylesheet" type="text/css"
      media="screen and (color)" href="example.css" />
```

- Note: the keyword "`all`" applies to all media types and is the default
- Here are two equivalent pair of media queries

```
<style>
@media all and (min-width:500px) {
  …
}
@media (min-width:500px) {
  …
}
</style>
```

Media Queries (cont'd)

- Enhanced media types allows targeting of specific physical characteristics of the device, e.g.

```
<link rel="stylesheet" type="text/css"
      media="screen and (max-device-width: 480px)" href="min.css" />
```

- A media type (screen), and
- the actual query enclosed within parentheses, containing a particular *media feature* (max-device-width) to inspect, followed by the target value (480px).
- the expression is asking the device if its horizontal resolution (max-device-width) is equal to or less than 480px.

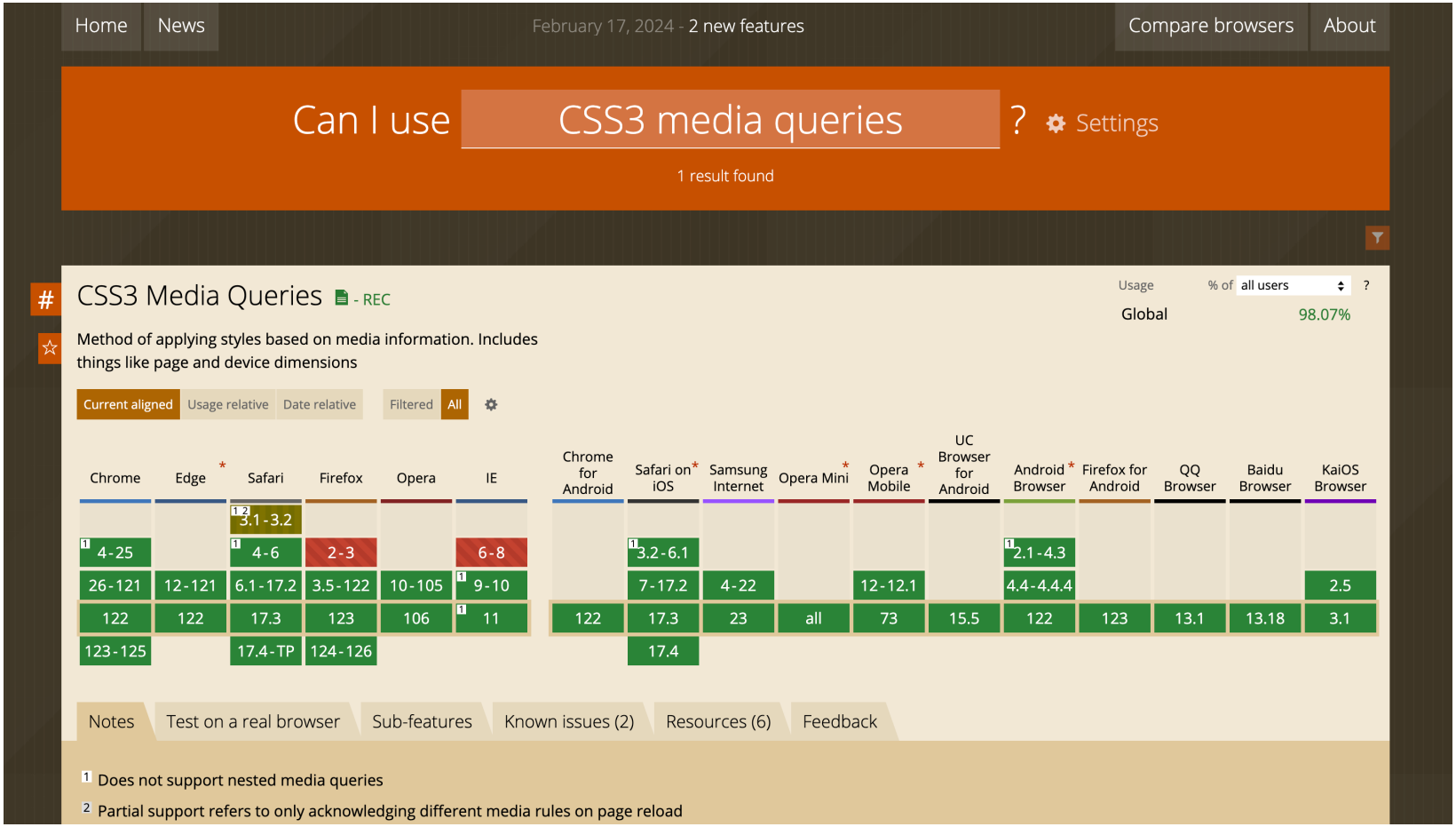
Media Queries (cont'd)

- Multiple property values in a single query can occur by chaining them together with the ***and*** keyword

```
<link rel="stylesheet" media="only screen and (min-width:200px) and (max-width: 500px)" href="small.css">
```

```
<link rel="stylesheet" media="only screen and (min-width:501px) and (max-width: 1100px)"
    href="large.css">
```

caniuse.com for Media Queries

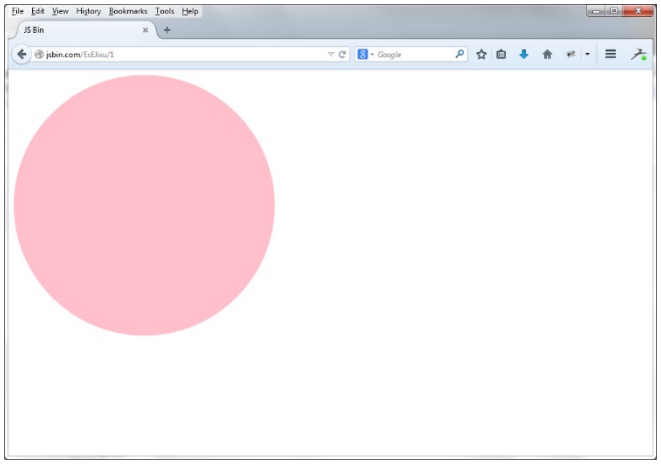


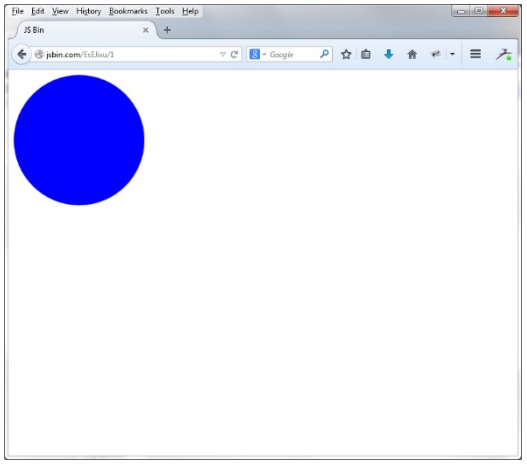
"Can I use" provides up-to-date browser support tables for front-end web technologies on desktop and mobile web browsers. The site was built and is maintained by Alexis Deveria".

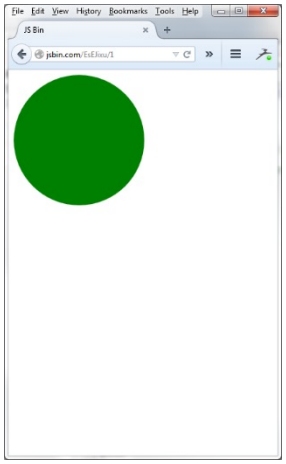
All recent browser versions support media queries.

A Simple Example

- Create a circle that is green for mobile phones, blue for tablets, and pink for desktops (watch with Chrome resizing)







find the code here http://jsbin.com/EsEJixu/1/

A Simple Example: the Source Code

```
<style>
.myCircle {
  width:200px;
  height:200px;
  -webkit-border-radius: 50%;
  -moz-border-radius: 50% ;
  border-radius: 50% ;
  background:blue;
}

@media (max-width: 480px) {
   .myCircle {
       background:red;
   }
}
```

```
@media (max-width: 768px) {
   .myCircle {
       background:green;
   }
}


@media (min-width: 960px) {
   .myCircle {
       background:pink;
       width:400px;
       height:400px;
   }
}
</style>
```

```
<body>
  <div class="myCircle"></div>
</body>
```

Note: to create a circle, width and height must be equal and border-radius set to 50%, which causes the to wrap around

Hiding Content on Smaller Screens

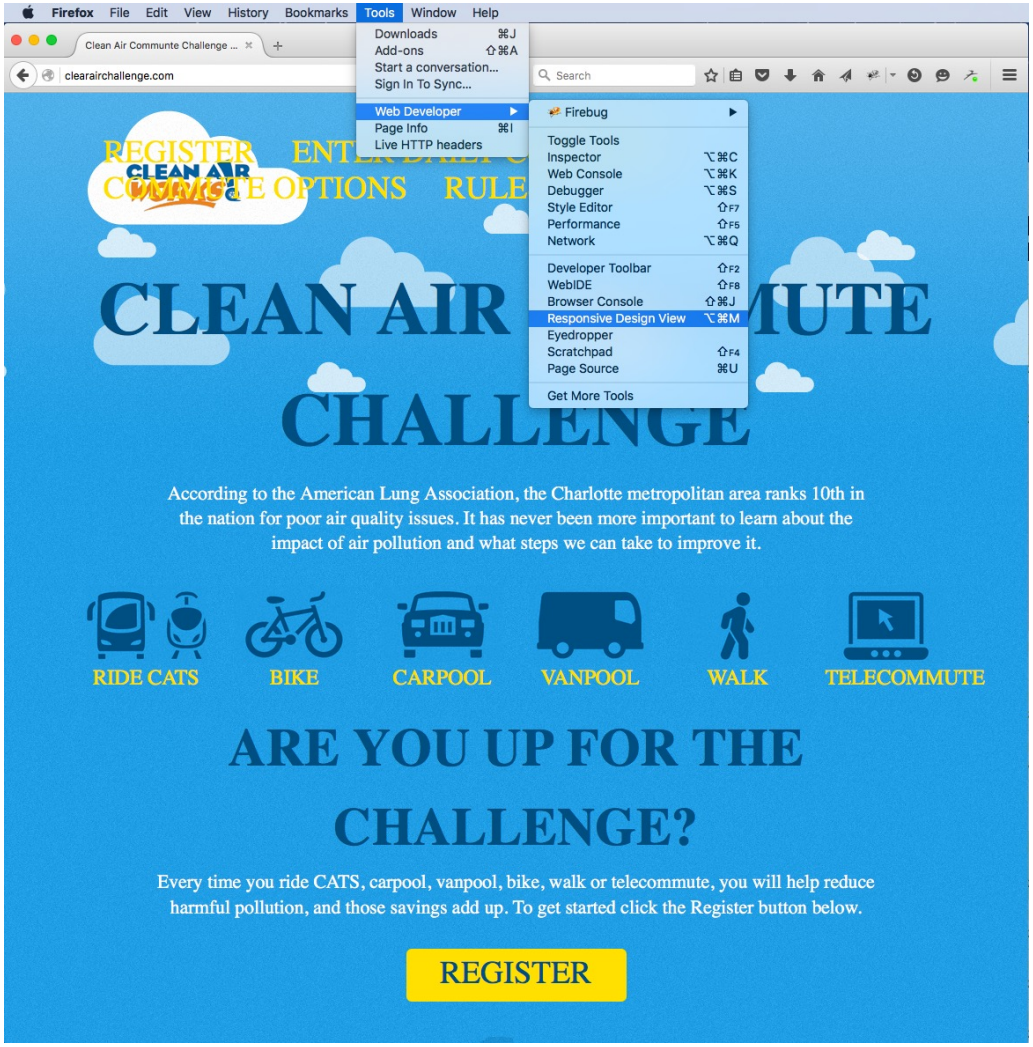
- In some cases, it may be advisable to hide content on the mobile device that would otherwise be visible on the desktop
  - this is done so users with small screens can avoid long scrolls
  - the usual way to handle this is to provide a link to "additional content"
- CSS allows us to show and hide content using

```
display: none;
```

- Either declare `display: none` for the HTML block element that needs to be hidden in a specific style sheet or detect the browser width and do it through JavaScript.
- **Note** that `visibility: hidden` just hides the content (although it is still there), whereas the `display` property gets rid of it altogether.
- See Google on Mobile FriendlyWebsites

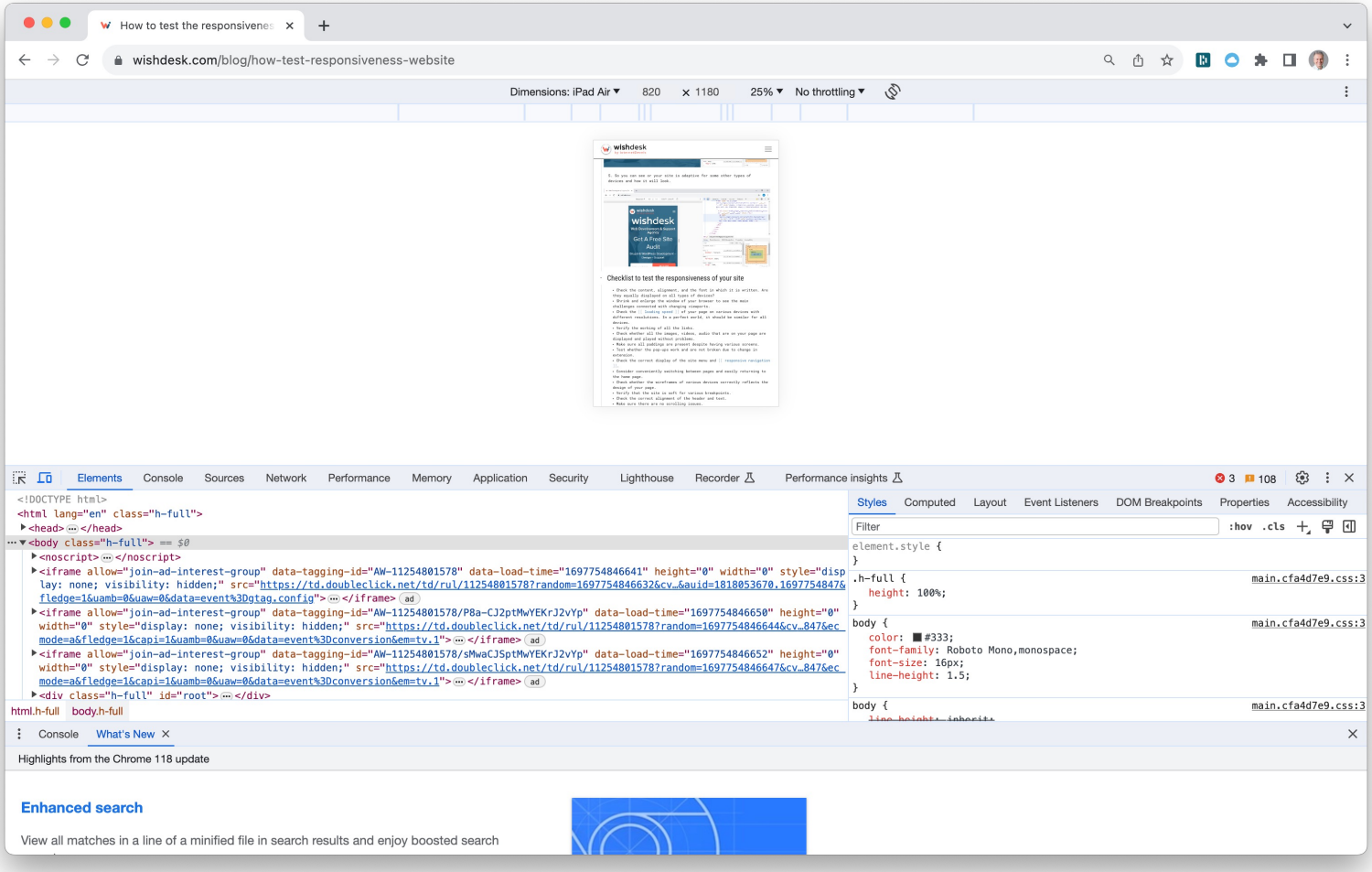
`https://developers.google.com/search/mobile-sites/`

Firefox Includes a Tool to Check for Responsive Design View



Chrome Includes a Tool to Check for Responsive Design View

Right click on landing page. Click **Inspect**. Click **Toggle device toolbar** button.



Creating Fluid Grids

- In "*adaptive grids*", we define pixel-based dimensions.
  - Hence, we have to adjust the widths and heights manually for certain device viewports.
- In "*fluid grids*" we define relative-based dimensions.
  - Since fluid grids flow naturally within the dimensions of its parent container, limited adjustments will be needed for various screen sizes and devices.
- In fluid grids we
  1. Define a maximum layout size for the design.
  2. The grid is divided into a specific number of columns to keep the layout clean and easy to handle.
  3. Then we design each element with proportional widths and heights instead of pixel-based dimensions.
- So, whenever the device or screen size is changed, elements will adjust their widths and heights by the specified proportions to its parent container.

Converting Fixed Pixel Sizes to Percentages

- To transform pixel-based column widths into percentage-based, *flexible* measurements use the formula: `target ÷ context = result`
- If the initial value of the page's title is 700px — but it's contained within a designed width of 988px, divide 700px (the target) by 988px (the context) like so:
- `700 ÷ 988 = 0.7085`
- And 0.7085 translates into 70.85%, a width one can drop directly into the stylesheet:

```
h1 { width: 70.85%;              /* 700px / 988px = 0.7085 */  }
```

- we can do the same with the margin of 144px (288px ÷ 2):
- `144 ÷ 988 = 0.14575`
- the 0.14575 becomes 14.575%, and add that directly to the style rule as a value for the title's margin-left:

```
h1 { margin-left: 14.575%;     /* 144px / 988px = 0.14575 */
     width: 70.85%;           /* 700px / 988px = 0.7085 */   }
```

Flexible Images

- To avoid having an image deformed due to the screen size one should avoid specific definitions of width and height and instead use CSS's `max-width` property setting it to 100%:

```
img { max-width: 100%; }
```

- As long as no other width-based image styles override this rule, every image will load in its original size, unless the viewing area becomes narrower than the image's original width.
  - With the maximum width of the image set to 100% of the screen or browser width, if the screen becomes narrower, so does the image.
- The browser will resize the images as needed using CSS to guide their relative size.

More Examples of Responsive Web Sites

For some additional examples of responsive web design see:

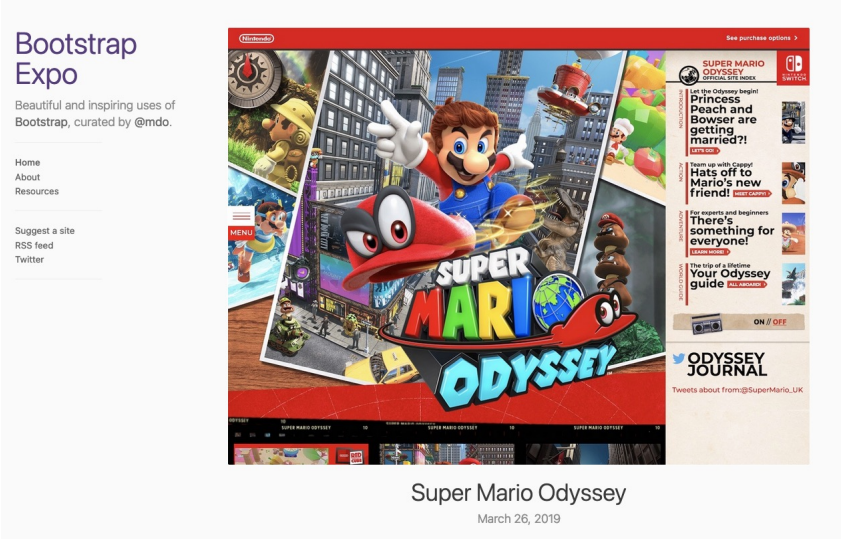
- http://hicksdesign.co.uk/
- http://2011.uxlondon.com/
- http://www.stpaulsschool.org.uk/
- http://css-tricks.com/
- http://yiibu.com/
- http://sickdesigner.com/
- http://ethanmarcotte.com/
- http://foodsense.is
- http://earthhour.fr
- http://w3conf.org
- http://mediaqueri.es
- http://fourkitchens.com
- http://achieveinternet.com

Bootstrap

- Bootstrap is a powerful, open source, front-end framework for faster and easier responsive web development.
- Originally named Twitter Blueprint, was developed by Mark Otto and Jacob Thornton at Twitter, as a framework for consistency across internal tools.
- It was renamed Bootstrap and released as open-source in 2011.
- It is still maintained by Otto, Thornton, and lots of contributors.
- It is used by 20% of all web sites.
- It includes HTML and CSS based design templates for common user interface components like Typography, Forms, Buttons, Tables, Navigations, Dropdowns, Alerts, Modals, Tabs, Accordion, Carousel and many other as well as optional JavaScript extensions.
- It gives you the ability to create responsive layout with much less effort.
- Its responsive features make web pages to appear more appropriately on different devices and screen resolutions without any change in markup.

Getting Started

- There are two versions available for download, **compiled Bootstrap** and **Bootstrap source** files. Latest version is 5.3.3.
- We recommend use stable **version 5.3.2**.
- Go to `http://getbootstrap.com/` and click on Download Bootstrap
- Select the compiled and minified CSS, JavaScript and fonts, or link to CDN
- Lots of different installations available
  - May need jQuery, Popper
- See samples of uses of Bootstrap at: `http://expo.getbootstrap.com`



Supported Browsers

- Specifically, **Bootstrap 4.6.x** supports the latest versions of the major browsers and platforms. On Windows, it fully supports Internet Explorer 11 and Edge. Partial support is available for IE 10.

Mobile devices

Generally speaking, Bootstrap supports the latest versions of each major platform's default browsers. Note that proxy browsers (such as Opera Mini, Opera Mobile's Turbo mode, UC Browser Mini, Amazon Silk) are not supported.

	Chrome	Firefox	Safari	Android Browser & WebView	Microsoft Edge
Android	Supported	Supported	N/A	Android v5.0+ supported	Supported
iOS	Supported	Supported	Supported	N/A	Supported
Windows 10 Mobile	N/A	N/A	N/A	N/A	Supported

Desktop browsers

Similarly, the latest versions of most desktop browsers are supported.

	Chrome	Firefox	Internet Explorer	Microsoft Edge	Opera	Safari
Mac	Supported	Supported	N/A	Supported	Supported	Supported
Windows	Supported	Supported	Supported, IE10+	Supported	Supported	Not supported

Supported Browsers

- Specifically, **Bootstrap 5.x** supports the latest versions of the major browsers and platforms.

Mobile devices

Generally speaking, Bootstrap supports the latest versions of each major platform's default browsers. Note that proxy browsers (such as Opera Mini, Opera Mobile's Turbo mode, UC Browser Mini, Amazon Silk) are not supported.

	Chrome	Firefox	Safari	Android Browser & WebView
Android	Supported	Supported	—	v6.0+
iOS	Supported	Supported	Supported	—

Desktop browsers

Similarly, the latest versions of most desktop browsers are supported.

	Chrome	Firefox	Microsoft Edge	Opera	Safari
Mac	Supported	Supported	Supported	Supported	Supported
Windows	Supported	Supported	Supported	Supported	—

For Firefox, in addition to the latest normal stable release, we also support the latest Extended Support Release (ESR) version of Firefox.

Unofficially, Bootstrap should look and behave well enough in Chromium and Chrome for Linux, and Firefox for Linux, though they are not officially supported.

The Bootstrap file structure (v4)

```
bootstrap/
├── css/
│   ├── bootstrap-grid.css
│   ├── bootstrap-grid.css.map
│   ├── bootstrap-grid.min.css
│   ├── bootstrap-grid.min.css.map
│   ├── bootstrap-reboot.css
│   ├── bootstrap-reboot.css.map
│   ├── bootstrap-reboot.min.css
│   ├── bootstrap-reboot.min.css.map
│   ├── bootstrap.css
│   ├── bootstrap.css.map
│   ├── bootstrap.min.css
│   └── bootstrap.min.css.map
└── js/
    ├── bootstrap.bundle.js
    ├── bootstrap.bundle.js.map
    ├── bootstrap.bundle.min.js
    ├── bootstrap.bundle.min.js.map
    ├── bootstrap.js
    ├── bootstrap.js.map
    ├── bootstrap.min.js
    └── bootstrap.min.js.map
```

Compiled CSS and JS files, bootstrap.* as well as Compiled and minified CSS and JS (bootstrap.min.*)
Reboot provides improved cross-browser rendering
Four font files (glyphicons-halflings-regular.* in the fonts folder)

CSS files	Layout	Content	Components	Utilities
`bootstrap.css` `bootstrap.min.css`	Included	Included	Included	Included
`bootstrap-grid.css` `bootstrap-grid.min.css`	Only grid system	Not included	Not included	Only flex utilities
`bootstrap-reboot.css` `bootstrap-reboot.min.css`	Not included	Only Reboot	Not included	Not included

JS files	Popper	jQuery
`bootstrap.bundle.js` `bootstrap.bundle.min.js`	Included	Not included
`bootstrap.js` `bootstrap.min.js`	Not included	Not included

The Bootstrap file structure (v5)

```
bootstrap/
├── css/
│   ├── bootstrap-grid.css
│   ├── bootstrap-grid.css.map
│   ├── bootstrap-grid.min.css
│   ├── bootstrap-grid.min.css.map
│   ├── bootstrap-grid.rtl.css
│   ├── bootstrap-grid.rtl.css.map
│   ├── bootstrap-grid.rtl.min.css
│   ├── bootstrap-grid.rtl.min.css.map
│   ├── bootstrap-reboot.css
│   ├── bootstrap-reboot.css.map
│   ├── bootstrap-reboot.min.css
│   ├── bootstrap-reboot.min.css.map
│   ├── bootstrap-reboot.rtl.css
│   ├── bootstrap-reboot.rtl.css.map
│   ├── bootstrap-reboot.rtl.min.css
│   ├── bootstrap-reboot.rtl.min.css.map
│   ├── bootstrap-utilities.css
│   ├── bootstrap-utilities.css.map
│   ├── bootstrap-utilities.min.css
│   ├── bootstrap-utilities.min.css.map
│   ├── bootstrap-utilities.rtl.css
│   ├── bootstrap-utilities.rtl.css.map
│   ├── bootstrap-utilities.rtl.min.css
│   ├── bootstrap-utilities.rtl.min.css.map
│   ├── bootstrap.css
│   ├── bootstrap.css.map
│   ├── bootstrap.min.css
│   ├── bootstrap.min.css.map
│   ├── bootstrap.rtl.css
│   ├── bootstrap.rtl.css.map
│   ├── bootstrap.rtl.min.css
│   └── bootstrap.rtl.min.css.map
└── js/
    ├── bootstrap.bundle.js
    ├── bootstrap.bundle.js.map
    ├── bootstrap.bundle.min.js
    ├── bootstrap.bundle.min.js.map
    ├── bootstrap.esm.js
    ├── bootstrap.esm.js.map
    ├── bootstrap.esm.min.js
    ├── bootstrap.esm.min.js.map
    ├── bootstrap.js
    ├── bootstrap.js.map
    ├── bootstrap.min.js
    └── bootstrap.min.js.map
```

CSS files

Bootstrap includes a handful of options for including some or all of our compiled CSS.

CSS files	Layout	Content	Components	Utilities
bootstrap.css bootstrap.min.css bootstrap.rtl.css bootstrap.rtl.min.css	Included	Included	Included	Included
bootstrap-grid.css bootstrap-grid.rtl.css bootstrap-grid.min.css bootstrap-grid.rtl.min.css	Only grid system	—	—	Only flex utilities
bootstrap-utilities.css bootstrap-utilities.rtl.css bootstrap-utilities.min.css bootstrap-utilities.rtl.min.css	—	—	—	Included
bootstrap-reboot.css bootstrap-reboot.rtl.css bootstrap-reboot.min.css bootstrap-reboot.rtl.min.css	—	Only Reboot	—	—

JS files

Similarly, we have options for including some or all of our compiled JavaScript.

JS Files	Popper
bootstrap.bundle.js bootstrap.bundle.min.js	Included
bootstrap.js bootstrap.min.js	-

Basic Bootstrap Template File (v3-v4)

Copy

```
<!doctype html>
<html lang="en">
  <head>
    <!-- Required meta tags -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

    <!-- Bootstrap CSS -->
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@4.6.0/dist/css/bootstrap.min.css" integrity="sha384-B0vP5xmATw1+K9

    <title>Hello, world!</title>
  </head>
  <body>
    <h1>Hello, world!</h1>

    <!-- Optional JavaScript; choose one of the two! -->

    <!-- Option 1: jQuery and Bootstrap Bundle (includes Popper) -->
    <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js" integrity="sha384-DfXdz2htPH0lsSSs5nCTpuj/zy4C+OGpamoFVy38MVBnE+IbbVYUev
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@4.6.0/dist/js/bootstrap.bundle.min.js" integrity="sha384-Piv4xVNRyMGpqkS2by6br4gNJ7

    <!-- Option 2: Separate Popper and Bootstrap JS -->
    <!--
    <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js" integrity="sha384-DfXdz2htPH0lsSSs5nCTpuj/zy4C+OGpamoFVy38MVBnE+IbbVYUev
    <script src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/umd/popper.min.js" integrity="sha384-9/reFTGAW83EW2RDu2S0VKaIzap3H66lZH
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@4.6.0/dist/js/bootstrap.min.js" integrity="sha384-+YQ4JLhjyBLPDQt//I+STsc9iw4uQqACv
    -->
  </body>
</html>
```

Basic Bootstrap Template File (v5)

Copy to clipboa

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap demo</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min
  </head>
  <body>
    <h1>Hello, world!</h1>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bun
  </body>
</html>
```

```
<script src="https://cdn.jsdelivr.net/npm/@popperjs/core@2.11.8/dist/umd/popper.min
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.min.js"
```

See: https://getbootstrap.com/docs/5.3/getting-started/introduction/

Bootstrap Grid System

- Bootstrap 3 introduced the responsive mobile first ***fluid grid system*** that scales up to 12 columns as the device or viewport size increases.
- **Bootstrap 3** provides **4-grid tier** classes for quickly making grid layouts for different types of devices like phones (xs), tablets (sm), desktops (md),and larger desktops (lg). For example,
  - you can use the ***.col-xs-*** class to create grid columns for extra small devices like smartphones;
  - similarly, the ***.col-sm-*** for small screen devices like tablets;
  - the ***.col-md-*** class for medium size devices like desktop, and
  - the ***.col-lg-*** for large desktop screens.
- **Bootstrap 4** provides simplified navigation, removes Glyphicons, adds reboot, improved grids (up to **5-grid tier**), flexbox layout, improved form control, improved browser support, new utility classes. See:

  https://blog.templatetoaster.com/bootstrap-3-vs-bootstrap-4-migrate-differences/
- **Bootstrap 5** provides **6-grid tier** (xs, sm, md, lg, xl, xxl)

Bootstrap 5 Grid System

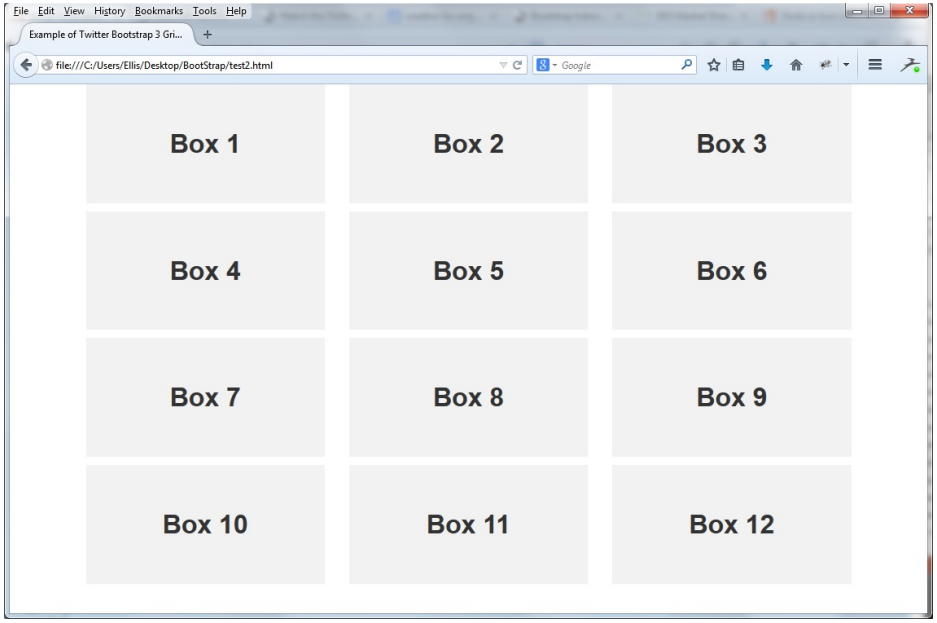
Difference between Bootstrap 4 and Bootstrap 5

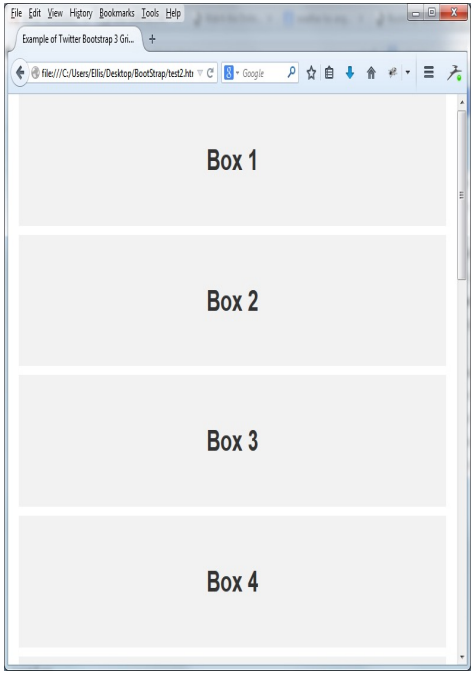
BASIS OF	BOOTSTRAP 4	BOOTSTRAP 5
Grid System	It has 5 tier (xs, sm, md, lg, xl).	It has 6 tier (xs, sm, md, lg, xl, xxl).
Color	It has limited colors.	Extra colors added with the looks,,A cardimproved color palette.
Jquery	It has jquery and all related plugins.	Jquery is removed and switched to vanilla JS with some working plugins
Internet Explorer	Bootstrap 4 supports both IE 10 and 11.	Bootstrap 5 doesn't support IE 10 and 11.
Form elements	Radio buttons, checkboxes have different look in different OS and browsers.	The look of form elements will not change, on different OS or browser.
Utilities API	We cannot modify utilities in bootstrap 4	Bootstrap 5 gave freedom to modify and also create our own utility
Gutter	We use .glutter with fontsize in px	We use .g* with fontsize in rem
Vertical Classes	Columns can be positioned relative	Columns cannot be positioned relative
Bootstrap Icons	Bootstrap 4 doesn't have its own SVG icons we have to font-awesome	Bootstrap 5 have its own SVG icons
Jumbotron	It supports.	It doesn't support jumbotron.
Card deck	The card deck is used to create an isset of cards with equal width and height.	Card deck class in removed in bootstrap
Navbar	We have inline-block property and we will get white dropdown as default for dropdown-menu-dark class.	Inline-block property is removed and we will get black dropdown as default for dropdown-menu-dark class.
Static Site Generator	Bootstrap 4 uses Jekyll software.	Bootstrap 5 uses Hugo software.

Creating Layouts with Bootstrap Grid System

- In the above illustration there are total 12 content boxes in all devices, but its placement vary according to the device screen size,
- E.g., the mobile device layout is rendered as one column grid layout , with 1 column and 12 rows placed above one another,
- in tablet it is rendered as two column grid layout which has 2 columns and 6 rows.
- in medium screen size devices like laptop and desktop it is rendered as three column grid layout which as 3 columns and 4 rows, and
- in large screen devices like large desktop, it is rendered as four column grid layout which has 4 columns and 3 rows.

Browser Output





In a laptop or desktop having screen or viewport width greater than or equal to 992px and less than 1200px;
it has 4 rows where each row has 3 equal columns resulting in 3x4 grid layout.

Bootstrap 4 Grid System

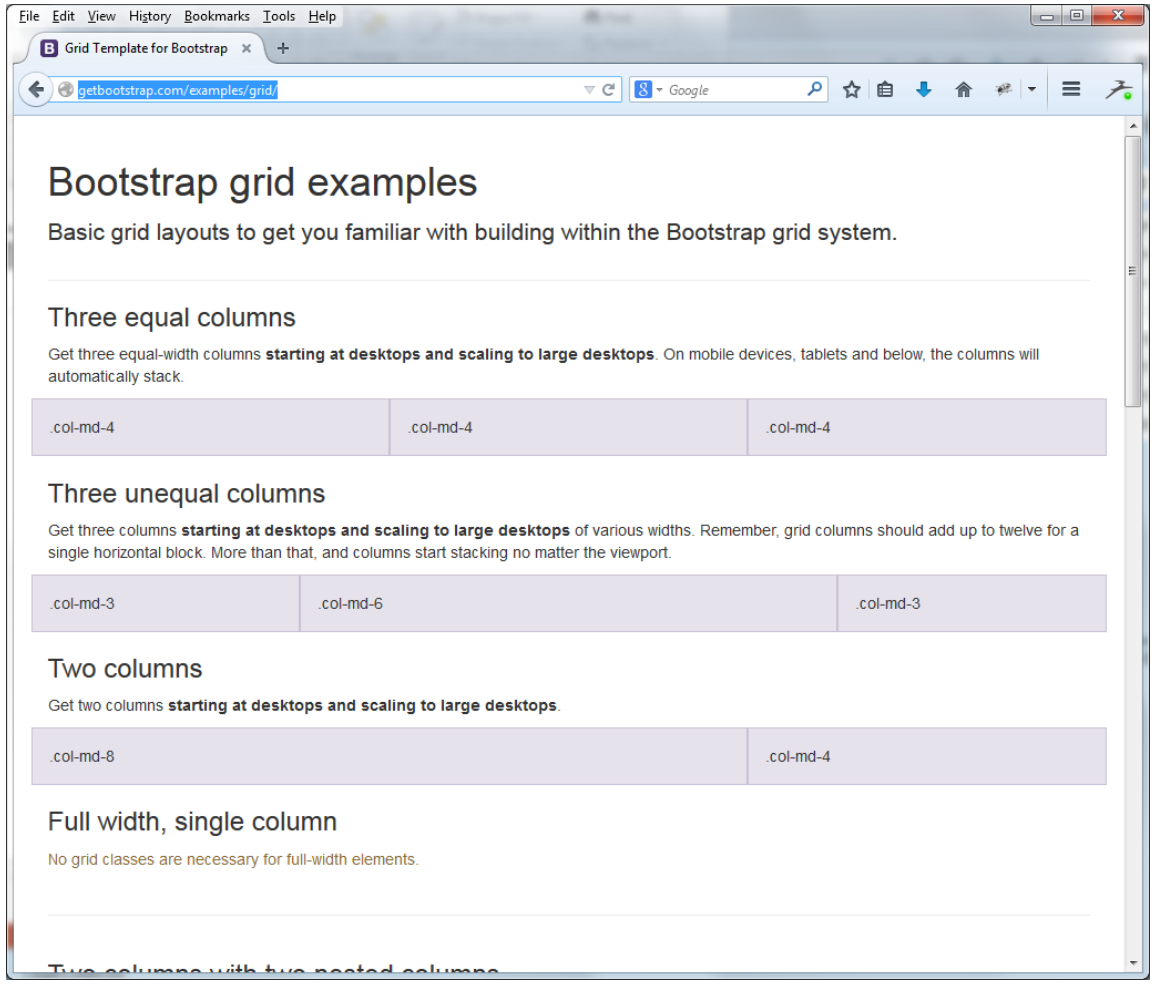
Applying any .col-sm- class to an element will not only affect its styling on small devices like tablets, but also on medium and large devices having screen size greater than or equal to 768px (i.e., ≥768px) if .col-md- and .col-lg- class is not present.

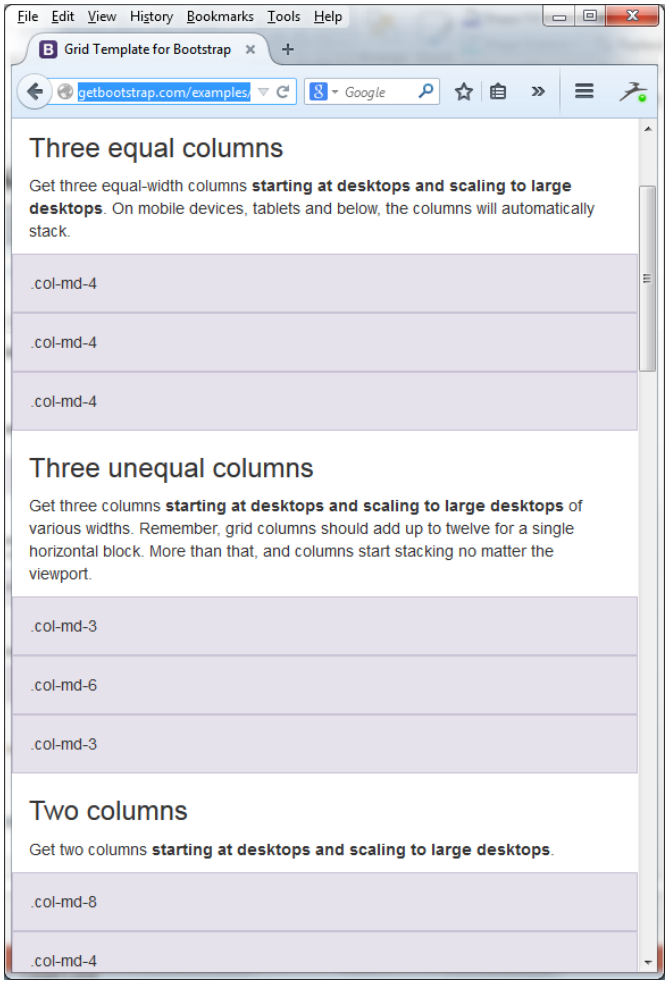
Similarly, the .col-md- class will not only affect the styling of elements on medium devices, but also on large devices if a .col-lg- class is not present

Grid columns should add **up to twelve columns** for a single horizontal block.
Bootstrap 4 changes Extra Small to <576, and shifts over to Extra Large, for 5 columns.

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	Extra large ≥1200px
Max container width	None (auto)	540px	720px	960px	1140px
Class prefix	`.col-`	`.col-sm-`	`.col-md-`	`.col-lg-`	`.col-xl-`
# of columns	12				
Gutter width	30px (15px on each side of a column)				
Nestable	Yes				
Column ordering	Yes				

Five Bootstrap Grid Examples



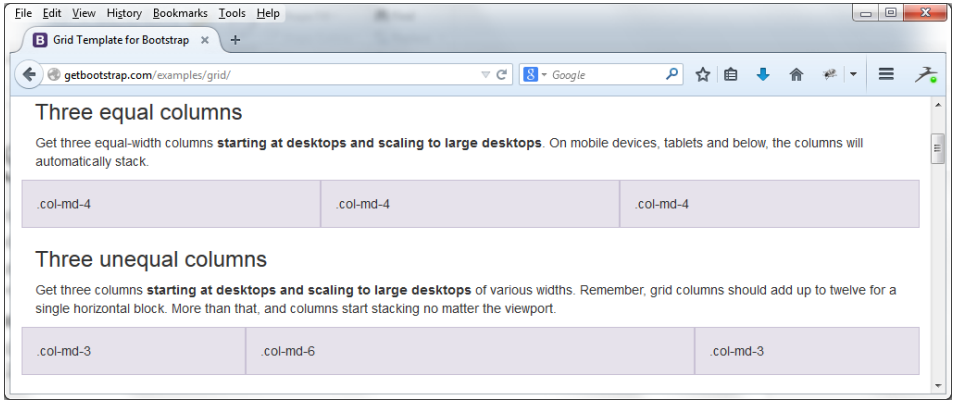


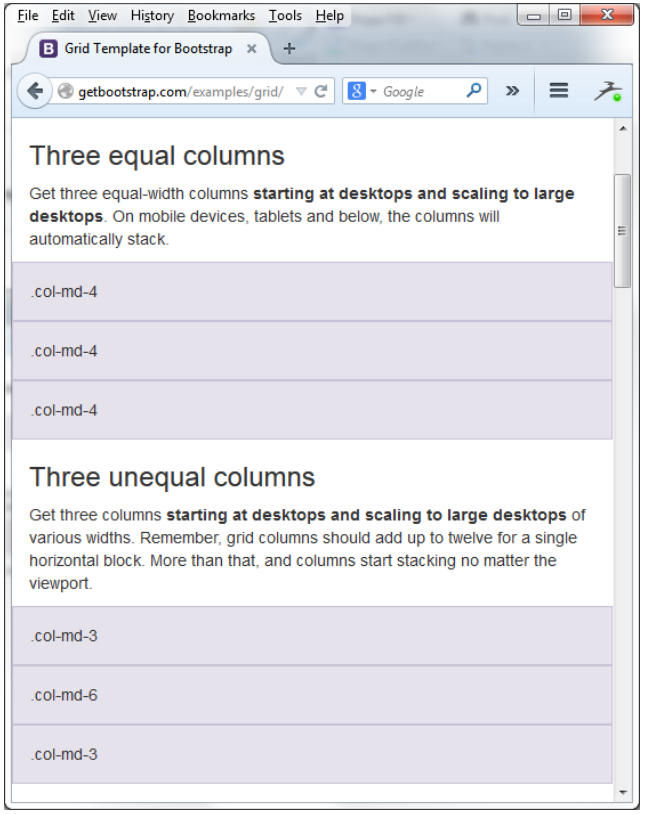
https://getbootstrap.com/docs/4.0/examples/grid/
https://getbootstrap.com/docs/4.6/layout/grid/ (4.6)

Source Code (1 of 5)

```
<!DOCTYPE html><html lang="en">
<head>
  <meta charset="utf-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="">
  <meta name="author" content="">
  <link rel="icon" href="../../favicon.ico">
  <title>Grid Template for Bootstrap</title>
  <!-- Bootstrap core CSS -->
  <link href="../../dist/css/bootstrap.min.css" rel="stylesheet">
  <!-- Custom styles for this template -->
  <link href="grid.css" rel="stylesheet">
  <!-- Just for debugging purposes. Don't actually copy these 2 lines! -->
  <!--[if lt IE 9]><script src="../../assets/js/ie8-responsive-file-
      warning.js"></script><![endif]-->
  <script src="../../assets/js/ie-emulation-modes-warning.js"></script>
  <!-- IE10 viewport hack for Surface/desktop Windows 8 bug -->
  <script src="../../assets/js/ie10-viewport-bug-workaround.js"></script>
```

Three Equal/Unequal Columns





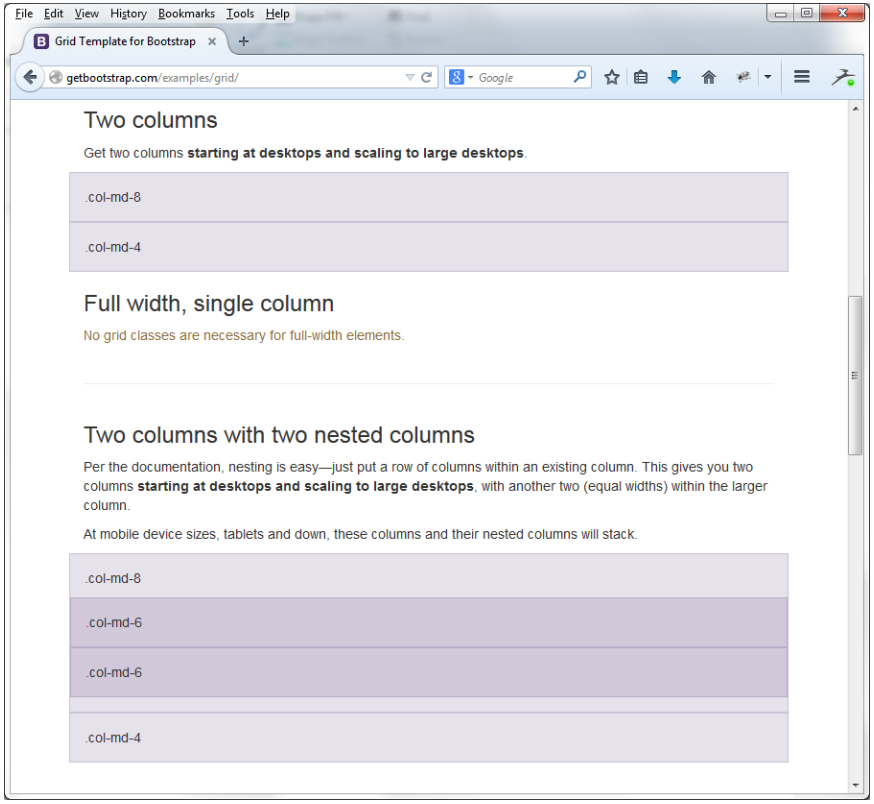
Source Code (2 of 5)

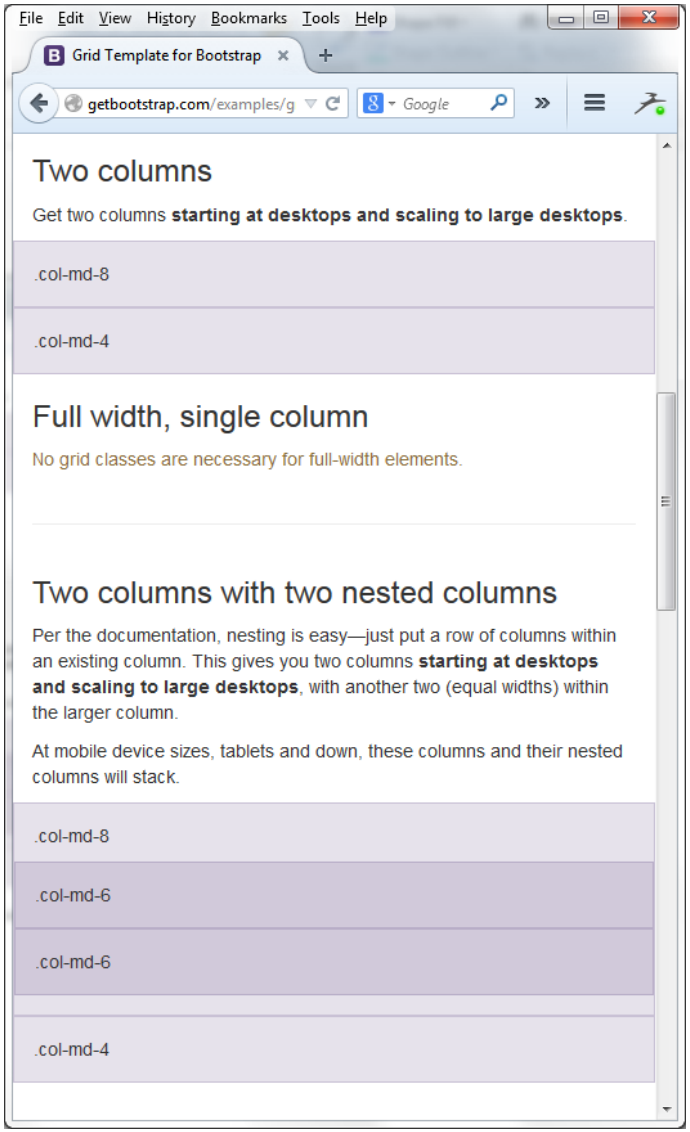
```
</head><body>
<div class="container">
  <div class="page-header">
    <h1>Bootstrap grid examples</h1>
    <p class="lead">Basic grid layouts to get you familiar with building within the
                    Bootstrap grid system.</p>
  </div>

  <h3>Three equal columns</h3>
  <p>Get three equal-width columns <strong>starting at desktops and scaling to large
     desktops</strong>. On mobile devices, tablets and below, the columns will
     automatically stack.</p>
  <div class="row">
    <div class="col-md-4">.col-md-4</div>
    <div class="col-md-4">.col-md-4</div>
    <div class="col-md-4">.col-md-4</div>
  </div>

  <h3>Three unequal columns</h3>
  <p>Get three columns <strong>starting at desktops and scaling to large
     desktops</strong> of various widths. Remember, grid columns should add up to twelve
     for a single horizontal block. More than that, and columns start stacking no matter
     the viewport.</p>
  <div class="row">
    <div class="col-md-3">.col-md-3</div>
    <div class="col-md-6">.col-md-6</div>
    <div class="col-md-3">.col-md-3</div>
</div>
```

Two Columns, Two with Nested Columns





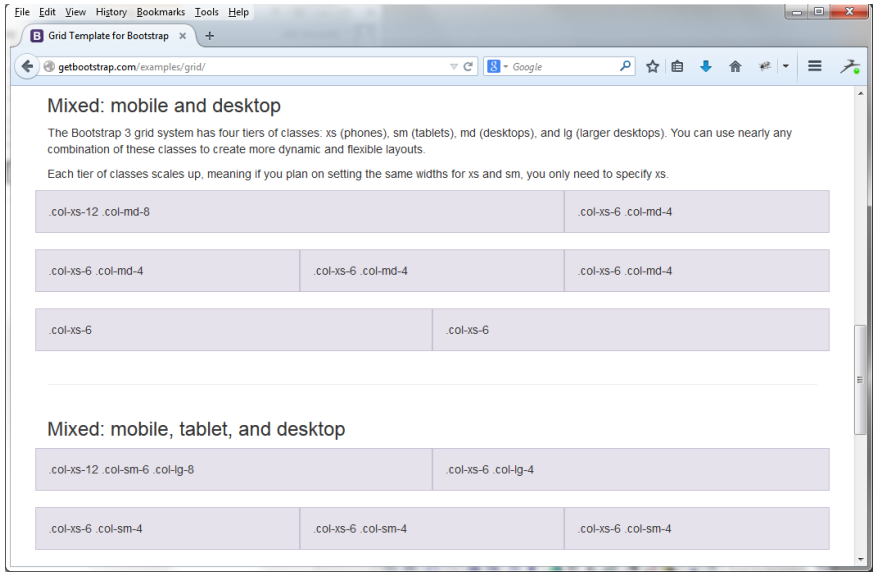
Source Code (3 of 5)

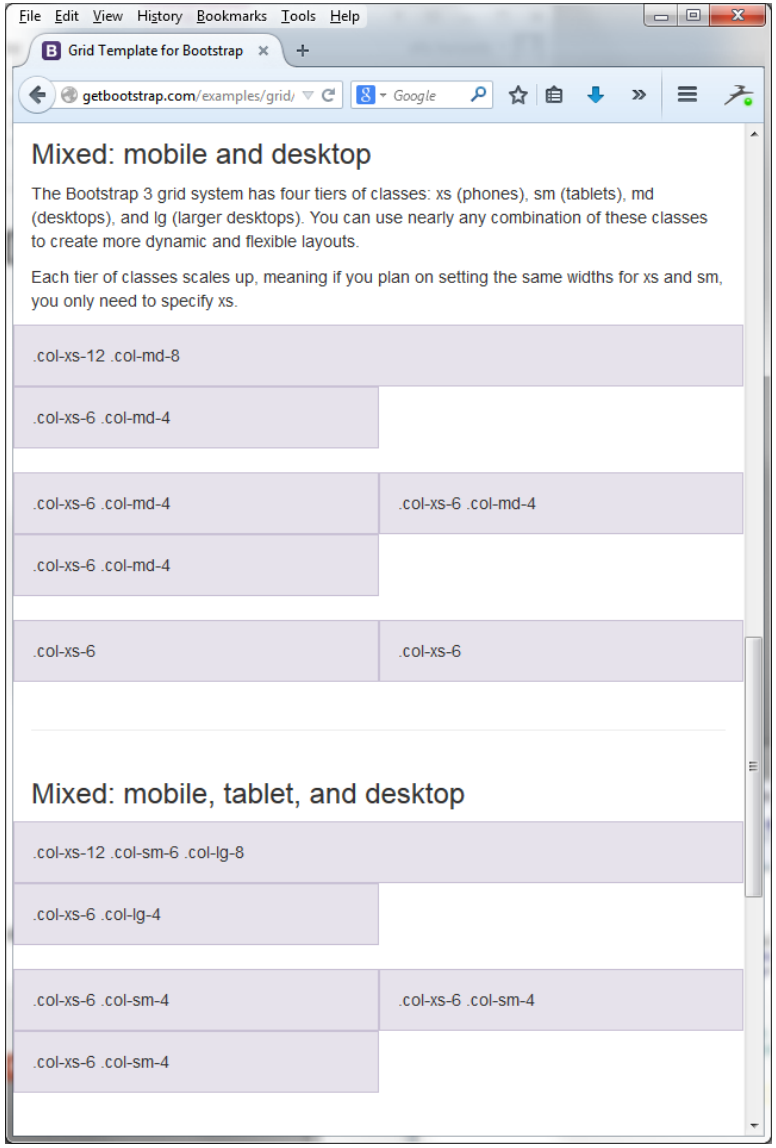
```
<h3>Two columns</h3>
<p>Get two columns <strong>starting at desktops and scaling to large desktops</strong>.</p>
<div class="row">
  <div class="col-md-8">.col-md-8</div>
  <div class="col-md-4">.col-md-4</div>
</div>

<h3>Full width, single column</h3>
<p class="text-warning">No grid classes are necessary for full-width elements.</p><hr>

<h3>Two columns with two nested columns</h3>
<p>Per the documentation, nesting is easy—just put a row of columns within an existing column.
   This gives you two columns <strong>starting at desktops and scaling to large desktops</strong>,
   with another two (equal widths) within the larger column.</p>
<p>At mobile device sizes, tablets and down, these columns and their nested columns will stack.</p>
<div class="row">
  <div class="col-md-8">
    .col-md-8
    <div class="row">
      <div class="col-md-6">.col-md-6</div>
      <div class="col-md-6">.col-md-6</div>
    </div>
  </div>
  <div class="col-md-4">.col-md-4</div>
</div>
<hr>
```

Mixed Mobile, Desktop, Tablet



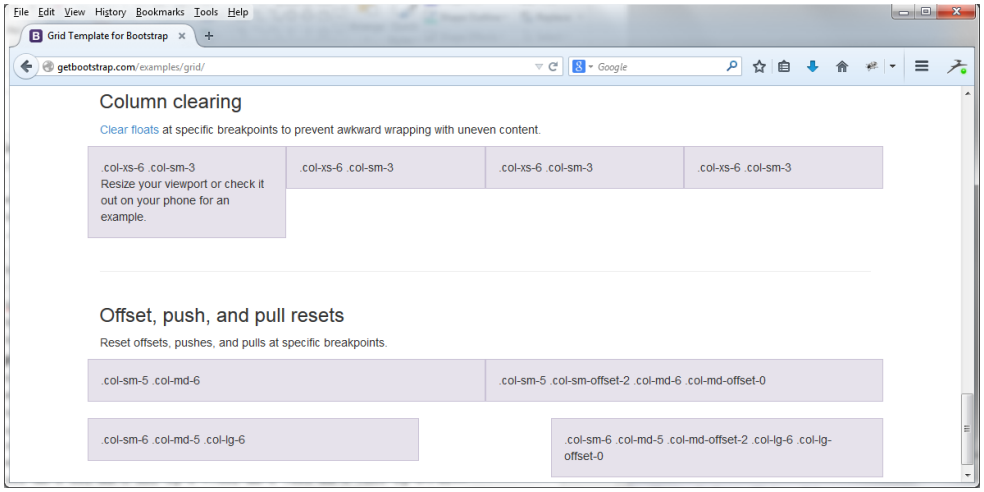


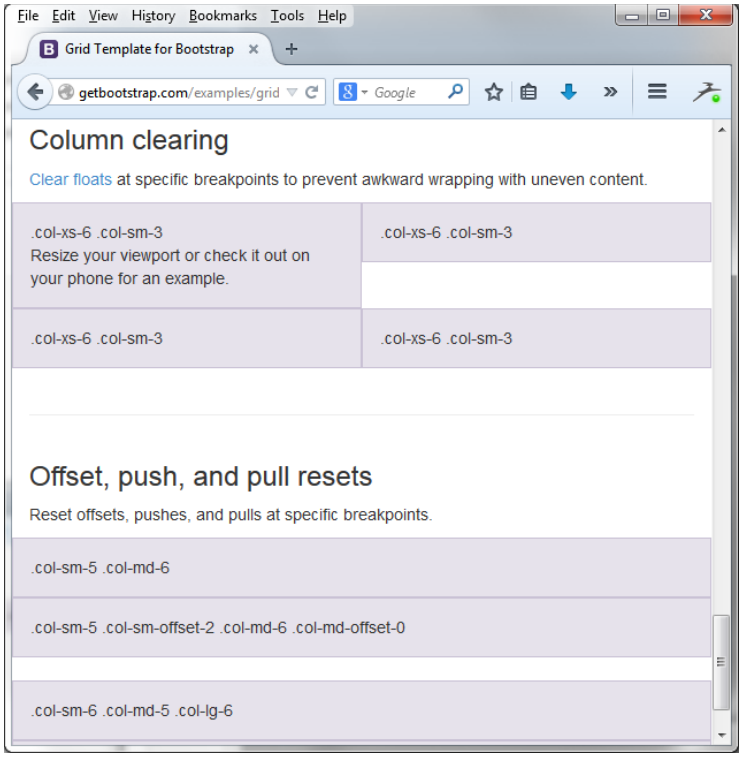
Source Code (4 of 5)

```
<h3>Mixed: mobile and desktop</h3>
<p>The Bootstrap 3 grid system has four tiers of classes: xs (phones), sm (tablets), md (desktops), and lg (larger desktops). You can use nearly any combination of these classes to create more dynamic and flexible layouts.</p>
<p>Each tier of classes scales up, meaning if you plan on setting the same widths for xs and sm, you only need to specify xs.</p>
<div class="row">
  <div class="col-xs-12 col-md-8">.col-xs-12 .col-md-8</div>
  <div class="col-xs-6 col-md-4">.col-xs-6 .col-md-4</div>
</div>
<div class="row">
  <div class="col-xs-6 col-md-4">.col-xs-6 .col-md-4</div>
  <div class="col-xs-6 col-md-4">.col-xs-6 .col-md-4</div>
  <div class="col-xs-6 col-md-4">.col-xs-6 .col-md-4</div>
</div>
<div class="row">
  <div class="col-xs-6">.col-xs-6</div>
  <div class="col-xs-6">.col-xs-6</div>
</div><hr>

<h3>Mixed: mobile, tablet, and desktop</h3>
<div class="row">
  <div class="col-xs-12 col-sm-6 col-lg-8">.col-xs-12 .col-sm-6 .col-lg-8</div>
  <div class="col-xs-6 col-lg-4">.col-xs-6 .col-lg-4</div>
</div>
<div class="row">
  <div class="col-xs-6 col-sm-4">.col-xs-6 .col-sm-4</div>
  <div class="col-xs-6 col-sm-4">.col-xs-6 .col-sm-4</div>
  <div class="col-xs-6 col-sm-4">.col-xs-6 .col-sm-4</div>
</div>
```

Column Clearing; Offset





Source Code (5 of 5)

```
<h3>Column clearing</h3>
<p><a href="http://getbootstrap.com/css/#grid-responsive-resets">Clear floats</a> at specific
breakpoints to prevent awkward wrapping with uneven content.</p>
<div class="row">
  <div class="col-xs-6 col-sm-3">
    .col-xs-6 .col-sm-3
    <br>Resize your viewport or check it out on your phone for an example.
  </div>
  <div class="col-xs-6 col-sm-3">.col-xs-6 .col-sm-3</div>
  <!-- Add the extra clearfix for only the required viewport -->
  <div class="clearfix visible-xs"></div>
  <div class="col-xs-6 col-sm-3">.col-xs-6 .col-sm-3</div>
  <div class="col-xs-6 col-sm-3">.col-xs-6 .col-sm-3</div>
</div><hr>

<h3>Offset, push, and pull resets</h3>
<p>Reset offsets, pushes, and pulls at specific breakpoints.</p>
<div class="row">
  <div class="col-sm-5 col-md-6">.col-sm-5 .col-md-6</div>
  <div class="col-sm-5 col-sm-offset-2 col-md-6 col-md-offset-0">
      .col-sm-5 .col-sm-offset-2 .col-md-6 .col-md-offset-0</div>
</div>
<div class="row">
  <div class="col-sm-6 col-md-5 col-lg-6">.col-sm-6 .col-md-5 .col-lg-6</div>
  <div class="col-sm-6 col-md-5 col-md-offset-2 col-lg-6 col-lg-offset-0">
     .col-sm-6 .col-md-5 .col-md-offset-2 .col-lg-6 .col-lg-offset-0</div>
</div></div> <!-- /container --><!-- Bootstrap core JavaScript -->
<!-- Placed at the end of the document so the pages load faster --></body></html>
```

Bootstrap 5 Grid System

See: https://getbootstrap.com/docs/5.3/layout/grid/

Grid options

Bootstrap's grid system can adapt across all six default breakpoints, and any breakpoints you customize. The six default grid tiers are as follow:

- Extra small (xs)
- Small (sm)
- Medium (md)
- Large (lg)
- Extra large (xl)
- Extra extra large (xxl)

As noted above, each of these breakpoints have their own container, unique class prefix, and modifiers. Here's how the grid changes across these breakpoints:

	xs <576px	**sm** ≥576px	**md** ≥768px	**lg** ≥992px	**xl** ≥1200px	**xxl** ≥1400px
Container `max-width`	None (auto)	540px	720px	960px	1140px	1320px
Class prefix	`.col-`	`.col-sm-`	`.col-md-`	`.col-lg-`	`.col-xl-`	`.col-xxl-`
# of columns	12					
Gutter width	1.5rem (.75rem on left and right)					
Custom gutters	Yes					
Nestable	Yes					
Column ordering	Yes					

Angular Bootstrap : ng-bootstrap

- “Angularized” Bootstrap widgets
- Built such a way that the only external dependency is the Bootstrap CSS
- Hence, no external JS is needed to run ng-bootstrap
- See: https://ng-bootstrap.github.io/#/home

// installation for Angular CLI (recommended)

```
ng add @ng-bootstrap/ng-bootstrap
```

ng-bootstrap Dependencies

The only two required dependencies are Angular and Bootstrap CSS. The supported versions are:

ng-bootstrap	Angular	Bootstrap CSS	Popper
	Show older versions		
10.x.x	^12.0.0	4.5.0	
11.x.x	^13.0.0	4.6.0	
12.x.x	^13.0.0	5.0.0	^2.10.2
13.x.x	^14.1.0	5.2.0	^2.10.2
14.x.x	^15.0.0	5.2.3	^2.11.6
15.x.x	^16.0.0	5.2.3	^2.11.6
16.x.x	^17.0.0	5.3.2	^2.11.8
17.x.x	^18.0.0	5.3.2	^2.11.8

ng-bootstrap – Node.js Dependencies

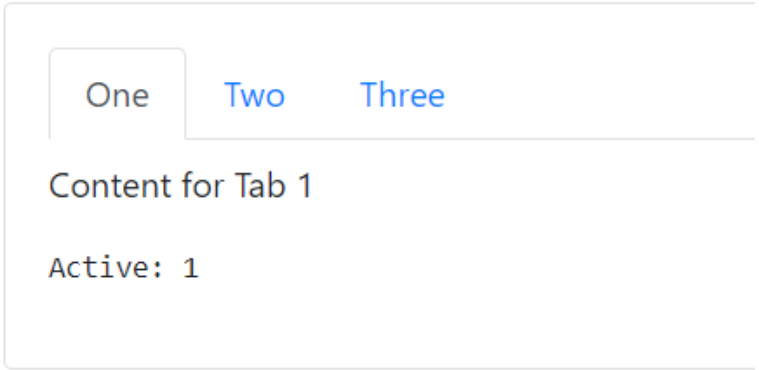
On Google Cloud only recent version are supported, especially Bootstrap 5.

Node	Angular	ng-bootstrap	Bootstrap CSS
Node.js 18	15.0.0	14.x.x	5.2.3
Node.js 18	16.0.0	15.x.x	5.2.3
Node.js 20	17.0.0	16.x.x	5.3.2

ng-bootstrap Components

ng-bootstrap Example : Nav

```
<ul ngbNav #nav="ngbNav" [(activeId)]="active" class="nav-tabs">
  <li [ngbNavItem]="1">
    <a ngbNavLink>One</a>
    <ng-template ngbNavContent>
      <p>Content For Tab 1</p>
    </ng-template>
  </li>
  <li [ngbNavItem]="2">
    <a ngbNavLink>Two</a>
    <ng-template ngbNavContent>
      <p>Content For Tab 2</p>
    </ng-template>
  </li>
  <li [ngbNavItem]="3">
    <a ngbNavLink>Three</a>
    <ng-template ngbNavContent>
      <p>Content For Tab 3</p>
    </ng-template>
  </li>
</ul>
<div [ngbNavOutlet]="nav" class="mt-2"></div>
<pre>Active: {{ active }}</pre>
```



https://stackblitz.com/run?file=app/nav-basic.ts

ng-bootstrap Example : Progress Bar

progressbar-striped.html progressbar-striped.ts

```
<p><ngb-progressbar type="success" [value]="25" [striped]="true"></ngb-progressbar></p>
<p><ngb-progressbar type="info" [value]="50" [striped]="true"></ngb-progressbar></p>
<p><ngb-progressbar type="warning" [value]="75" [striped]="true"></ngb-progressbar></p>
<p><ngb-progressbar type="danger" [value]="100" [striped]="true"></ngb-progressbar></p>
```

ng-bootstrap Example : Table

```
<table class="table table-striped">
  <thead>
  <tr>
    <th scope="col">#</th>
    <th scope="col">Country</th>
    <th scope="col">Area</th>
    <th scope="col">Population</th>
  </tr>
  </thead>
  <tbody>
  <tr *ngFor="let country of countries; index as i">
    <th scope="row">{{ i + 1 }}</th>
    <td>
      <img [src]="'https://upload.wikimedia.org/wikipedia/commons/' + country.flag"
      class="mr-2" style="width: 20px">
      {{ country.name }}
    </td>
    <td>{{ country.area | number }}</td>
    <td>{{ country.population | number }}</td>
  </tr>
  </tbody>
</table>
```

#	Country	Area	Population
1	Russia	17,075,200	146,989,754
2	Canada	9,976,140	36,624,199
3	United States	9,629,091	324,459,463
4	China	9,596,960	1,409,517,397

ng-bootstrap Example : Alert

Closable Alert

```
<p *ngFor="let alert of alerts">
  <ngb-alert [type]="alert.type" (closed)="close(alert)">{{ alert.message }}</ngb-alert>
</p>
<p>
  <button type="button" class="btn btn-primary" (click)="reset()">Reset</button>
</p>
```


```
@Component({
  selector: 'ngbd-alert-closeable',
  templateUrl: './alert-closeable.html'
})
export class NgbdAlertCloseable {

  alerts: Alert[];

  constructor() {
    this.reset();
  }

  close(alert: Alert) {
    this.alerts.splice(this.alerts.indexOf(alert), 1);
  }

  reset() {
    this.alerts = Array.from(ALERTS);
  }
}
```

ng-bootstrap Example : Self Closing Alert

```
<ngb-alert #selfClosingAlert *ngIf="successMessage" type="success" (closed)="successMessage = ''">{{
successMessage }}
</ngb-alert>
<p>
  <button class="btn btn-primary" (click)="changeSuccessMessage()">Change message</button>
</p>
```

```
export class NgbdAlertSelfclosing implements OnInit {
  private _success = new Subject<string>();

  staticAlertClosed = false;
  successMessage = '';

  @ViewChild('staticAlert', {static: false}) staticAlert: NgbAlert;
  @ViewChild('selfClosingAlert', {static: false}) selfClosingAlert: NgbAlert;

  ngOnInit(): void {
    setTimeout(() => this.staticAlert.close(), 20000);

    this._success.subscribe(message => this.successMessage = message);
    this._success.pipe(debounceTime(5000)).subscribe(() => {
      if (this.selfClosingAlert) {
        this.selfClosingAlert.close();
      }
    });
  }

  public changeSuccessMessage() { this._success.next(`${new Date()} - Message successfully changed.`);
  }
}
```

This shows self-closing success message that disappears after 5 seconds